

# Program 10: Demonstrate Kubernetes Dashboard and CLI for Cluster Monitoring

## Aim

To monitor and manage a Kubernetes cluster using both the **Kubernetes Dashboard (Web UI)** and the **Command Line Interface (kubectl)**.

---

## Description

This experiment demonstrates how to monitor and manage a Kubernetes cluster using the Kubernetes Dashboard and CLI tools. The Dashboard provides a graphical interface to visualize and interact with cluster resources, while the CLI offers powerful commands to inspect, debug, and manage resources efficiently.

---

## Project Structure

```
Program-10/  
├── deployment.yaml  
└── service.yaml
```

---

## Prerequisites

- Minikube installed
  - kubectl configured
  - Basic understanding of Kubernetes resources
- 

## Step 1: Start Minikube with Specific Resources

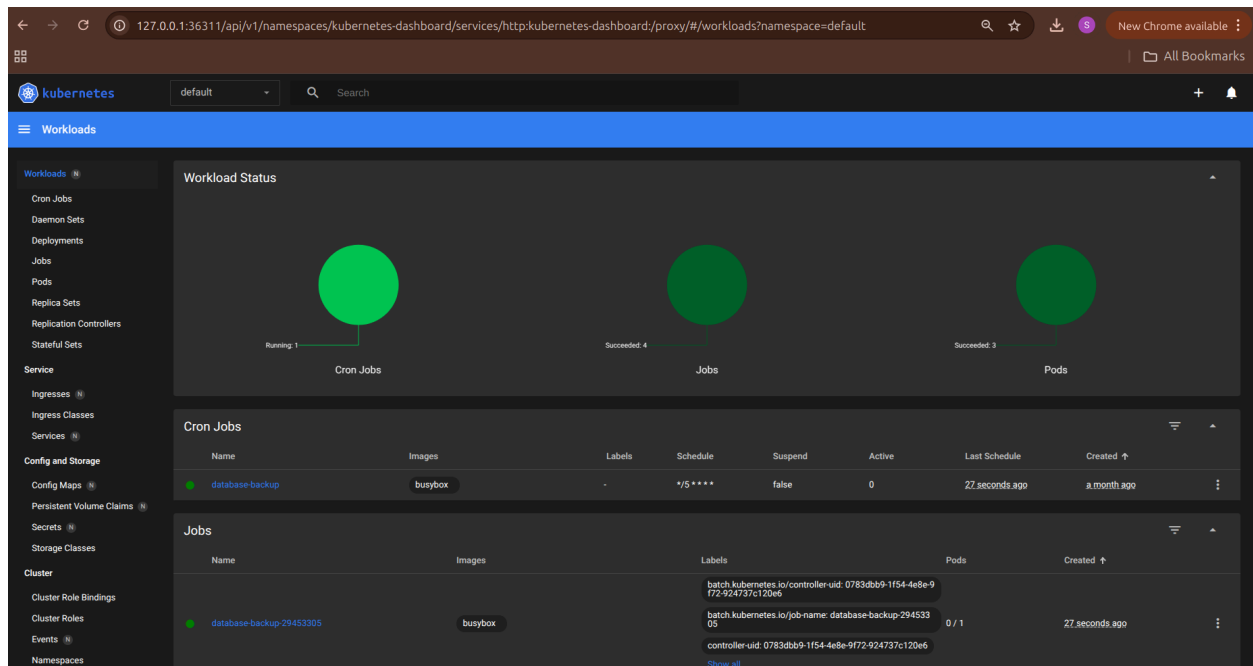
```
minikube start --cpus=2 --memory=4096
```

## Explanation

- Starts a local Kubernetes cluster
  - Allocates 2 CPUs and 4 GB RAM
- 

## Step 2: Enable Kubernetes Dashboard

minikube dashboard



## Dashboard Features

- View nodes, pods, deployments, and services
  - Inspect pod logs and events
  - Deploy applications from UI
- 

## Step 3: Create Deployment YAML (deployment.yaml)

apiVersion: apps/v1

kind: Deployment

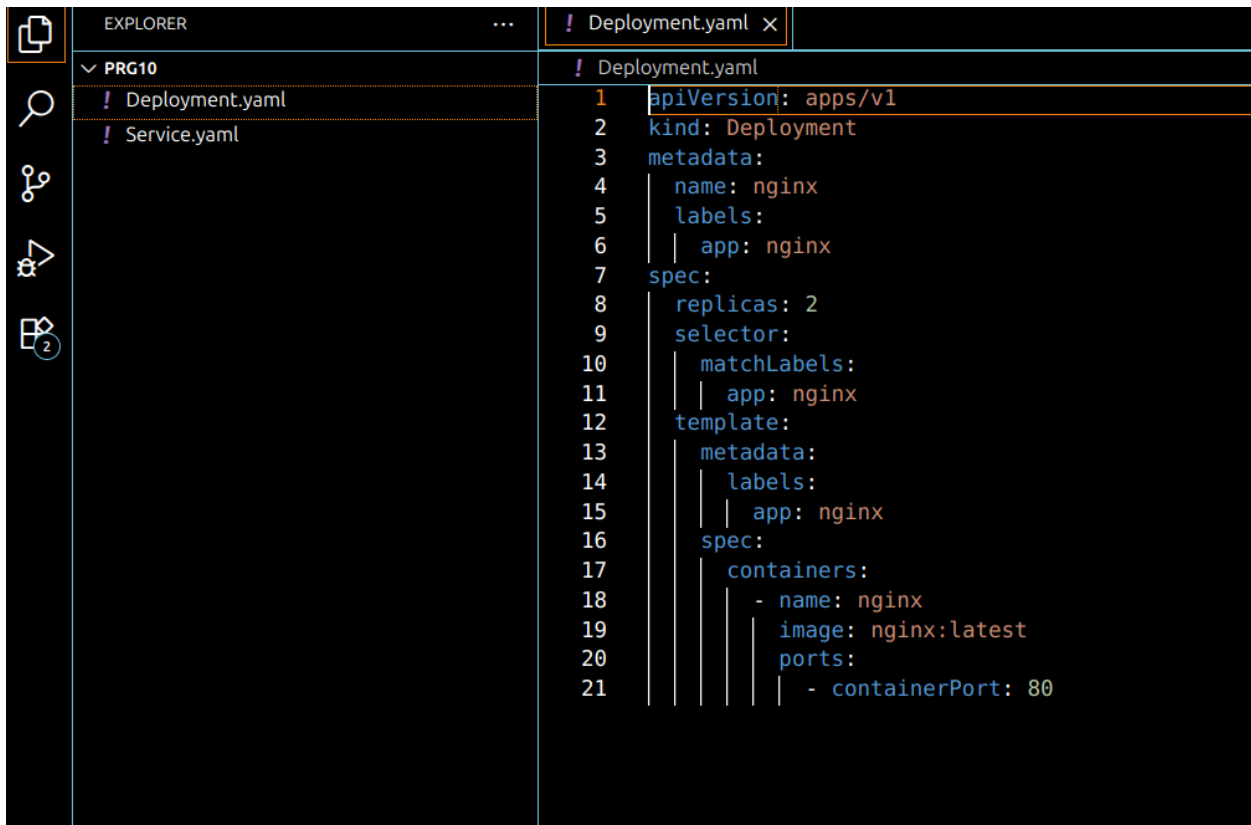
metadata:

name: nginx-deployment

labels:

app: nginx

```
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
```



The screenshot shows the Visual Studio Code interface with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'PRG10' containing two files: 'Deployment.yaml' and 'Service.yaml'. The code editor displays the content of 'Deployment.yaml', which is a Kubernetes Deployment manifest. The manifest includes fields for 'apiVersion', 'kind', 'metadata' (with name and labels), 'spec' (with replicas, selector, and template), and 'template' (with metadata, labels, and spec). The spec defines a single container named 'nginx' using the 'nginx:latest' image and exposing port 80.

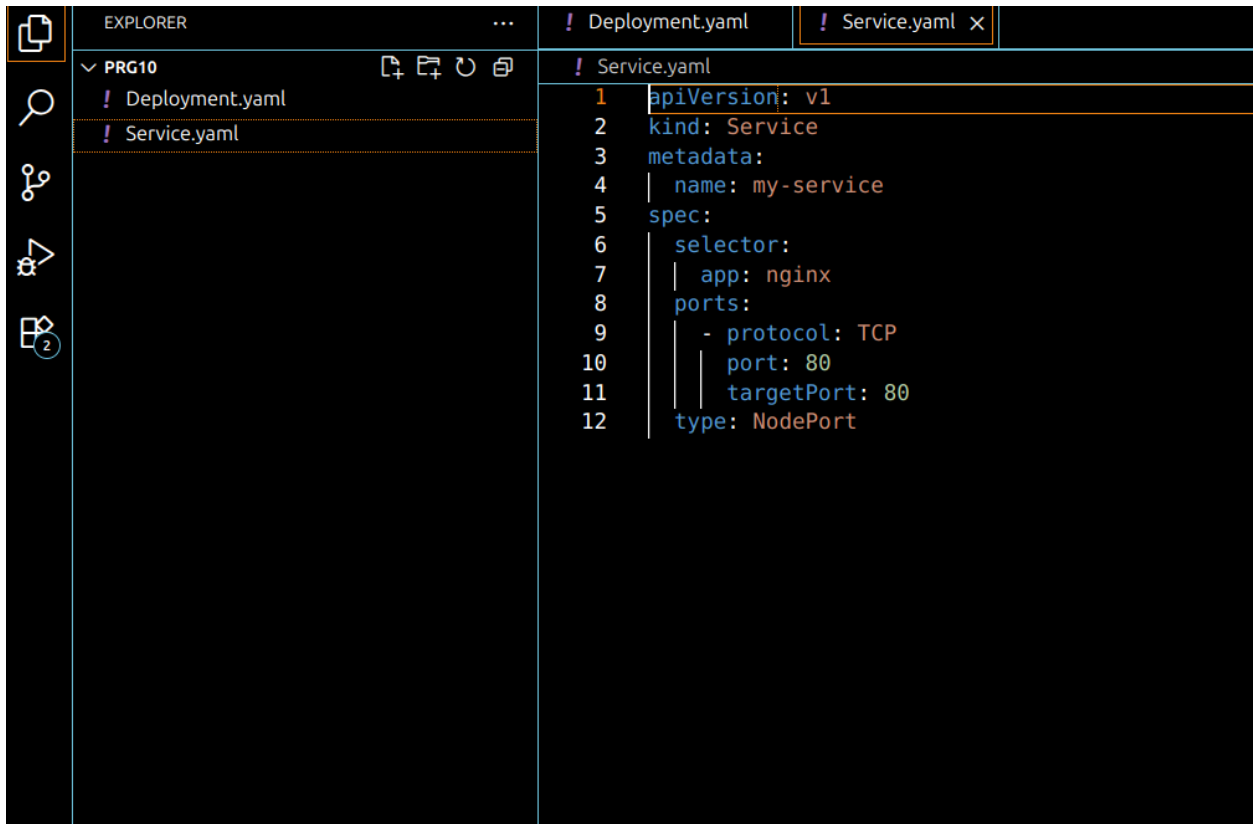
```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: nginx
5   labels:
6     app: nginx
7 spec:
8   replicas: 2
9   selector:
10    matchLabels:
11      app: nginx
12   template:
13     metadata:
14       labels:
15         app: nginx
16     spec:
17       containers:
18       - name: nginx
19         image: nginx:latest
20         ports:
21         - containerPort: 80
```

## Explanation

- Deployment manages multiple Pods
  - replicas ensures high availability
  - template defines Pod configuration
-

## Step 4: Create Service YAML (service.yaml)

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: NodePort
```

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project named 'PRG10' with two files: 'Deployment.yaml' and 'Service.yaml'. The 'Service.yaml' file is selected and its content is displayed in the main editor area. The code is as follows:

```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: my-service
5 spec:
6   selector:
7     app: nginx
8   ports:
9     - protocol: TCP
10       port: 80
11       targetPort: 80
12   type: NodePort
```

## Explanation

- Service exposes the application
  - NodePort allows external access
  - selector connects Service to Pods
-

## Step 5: Apply Kubernetes Resources

```
kubectl apply -f deployment.yaml  
kubectl apply -f service.yaml
```

---

## Step 6: Monitor via CLI

```
kubectl get pods  
kubectl get nodes  
kubectl get svc
```

---

## Step 7: Check Pod Logs

```
kubectl logs <pod-name>
```

---

## Step 8: Describe Pod

```
kubectl describe pod <pod-name>
```

---

## Step 9: Access the Application

```
minikube service nginx-service
```

The NGINX welcome page will open in the browser.

---

## Step 10: Monitor Using Dashboard

- View deployment status
  - Monitor pod health
  - Inspect logs and events
-

## Step 11: Clean-up

```
kubectll delete pods --all  
kubectll delete svc --all  
kubectll delete deployments --all
```

---

## Expected Output

- NGINX Pods running successfully
  - Service exposed via NodePort
  - Application accessible in browser
  - Cluster resources visible in Dashboard
- 

## Conclusion

This experiment demonstrates effective cluster monitoring using Kubernetes Dashboard and CLI commands by deploying and exposing an NGINX application.